Data-based modeling of slug flow crystallization with uncertainty quantification

1th Elisabeth Bünz Matr.: 218640
2th Paul Jollet Matr.: 218346
3th Rafael Kickartz Matr.: 226310
4th Gina Lüker Matr.: 220478
5th Ahmed Tantawy Matr.: 269602

***Abstract*—This report presents a data-driven modeling approach for a crystallization process in slug flow, combining exploratory data analysis, unsupervised learning, neural sequence modeling, and uncertainty quantification. After visual inspection and correlation analysis of experimental trajectory data, dbscan clustering was used to identify consistent process regimes. A NARX neural network was then trained on lagged input-output sequences to capture the system's dynamic behavior, using optimized hyperparameters obtained via Bayesian search. To quantify predictive uncertainty, conformalized quantile regression (CQR) was applied, reusing the NARX architecture to generate robust prediction intervals. The final model was evaluated under realistic closed-loop conditions in the Beat-The-Felix benchmark, demonstrating strong predictive performance and stability across all target variables.**

I. Introduction

This report presents the final results and insights obtained throughout the course of the project. The primary objective of this study is to develop a data-based model of a slug flow crystallizer with uncertainty quantification. This will be achieved by combining statistical analysis and machine learning techniques. The following sections offer a synopsis of the methodology, key findings, and implications of the study.

II. Results and Discussion

A. Loading, Analyzing and Visualizing the Data (Ahmed Tantawy)

To gain an initial understanding of the dataset, a comprehensive script was developed to handle data input, basic analysis, and exploratory visualization. All available data files are automatically loaded and merged into a single DataFrame. The script performs several key tasks:

- Computes summary statistics and checks for missing values
- Generates histograms and boxplots to inspect feature distributions
- Constructs a correlation matrix for all relevant numerical features
- Creates pairwise scatter plots for selected variable combinations

Although various visualizations were produced during this step, only the correlation matrix is included in this report, as it provides valuable information on the linear relationships between key process variables (see Figure 1). Strong positive correlations are observed, for example, between the mass flows and the stream temperatures, as well as between the input variables. The diameters also show strong correlation, indicating consistent behavior in the particle size distribution. Correlations values near zero help identify features that vary independently, which is useful for later modeling stages.

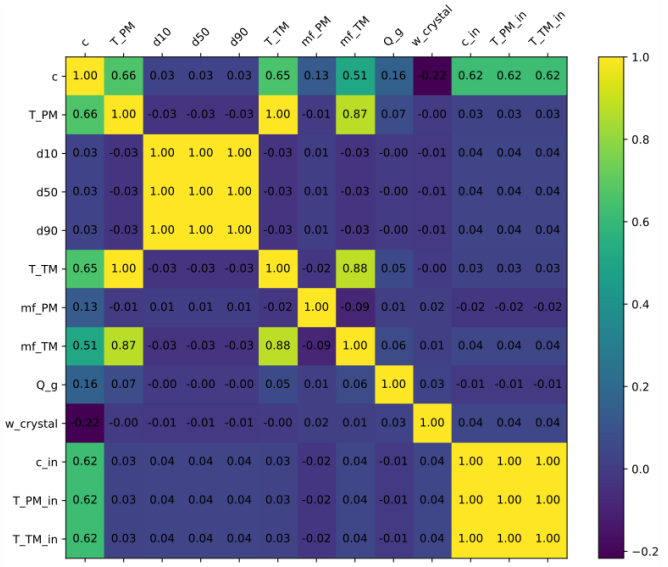

Fig. 1. Correlation Matrix for every variable pair, including input and state variables.

B. Clustering (Rafael Kickartz)

To identify distinct patterns within the dataset, unsupervised clustering was performed using the dbscan algorithm, based on two principal components extracted via PCA from sklearn. The parameters of the DBSCAN algorithm are set to a radius of $r = 0.3$ and a minimum sample size of five. This approach enables the visualization of the data in two dimensions, as illustrated in Figure 2.

In order to validate the physical plausibility of the data, it is imperative to examine a representative data point from each cluster. This step ensures that clustering is both mathematically sound and meaningful from a process perspective. Therefore, it is necessary to remove extraneous data from the data frame. Specifically, this entails the elimination of noise, clusters 2 and cluster 3. It is evident that clusters 2 and 3 are comprised of data points that are indicative of a malfunction in the sensor utilized for the detection of crystal size.

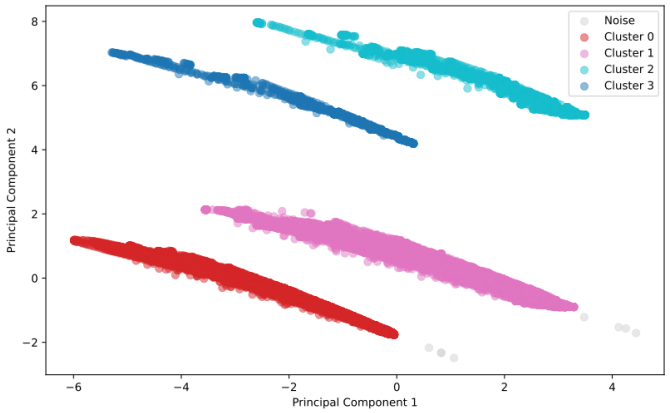

Fig. 2. Clustering results of the data using the dbscan algorithm. The initial 13 dimensions are condensed into two principal components, thereby enabling the illustration in two dimensions.

C. Data Splitting (Paul Jollet)

A crucial step in the modeling workflow is the proper separation of data into training, validation, and test sets. Because the NARX model uses lagged time windows as inputs (see Chapter II-D), individual samples cannot be selected at random. Instead, data must be split at the level of complete trajectories to avoid false information caused by overlapping time steps or randomly sorted data points. Furthermore, to ensure that the model generalizes across the different behavioral modes identified during clustering, the splitting was performed independently within each cluster. Clusters containing more than three trajectories are split in 70 % training, 20 % validation and 10 % testing data. In the case of smaller clusters, trajectories are sorted into sets based on the number of trajectories. The initial trajectory will be utilized for training purposes, while the second trajectory will be employed for validation. In the event that a third trajectory is available, it will be utilized for testing purposes.

D. Nonlinear AutoRegressive with eXogenous inputs (NARX) Model (Paul Jollet)

To predict key process variables over time, a NARX (Nonlinear AutoRegressive with eXogenous inputs) neural network was implemented using PyTorch. This model structure is particularly well-suited for time series forecasting, as it incorporates both past input and output values to model complex temporal dependencies. The pipeline consists of several stages, including data splitting, lagged sequence preparation, model training, and evaluation.

The implemented NARX model is a flexible feedforward neural network that accepts any size of lagged sequences of input and output variables. The input features include the lagged states and the lagged inputs, while the output targets are represented by the states of the next time step. The network architecture is designed to accommodate a variable number of hidden layers, with a variable number of neurons per layer. It incorporates dropout regularization to mitigate overfitting, configurable activation functions, a configurable number of learning epochs, the batch size and learning rate regularization. The Adam optimizer is used for training due to its efficiency and adaptability in handling sparse gradients and noisy data. The discrepancy between the predictions and the true values is quantified by the mean squared error (MSE). In the subsequent chapter, an explanation will be provided regarding the implementation of a Bayesian optimization algorithm for the purpose of optimizing the listed hyperparameters of the NARX model.

To identify the most effective model configuration, a Bayesian optimization strategy using the Optuna framework was employed. The optimizer minimizes the MSE for the validation set over 50 trials, testing a range of hyperparameters listed in Table I.

TABLE I
RANGE OF HYPERPARAMETERS FOR THE BAYESIAN OPTIMIZATION OF THE NARX MODEL.

Hyperparameter	Range
Lag	Integer in between 3 and 8
Hidden Layers	[64], [128], [64, 128], [128, 64], [128, 256, 128], [64, 128, 64]
Dropout	0 to 0.5
Activation	ReLU or tanh
Epochs	Steps of ten in between 50 and 100
Batch Size	16, 32, 64
Learning Rate	0.01 to 10^{-5}

The configuration that demonstrates optimal performance is characterized by a lag of 5, resulting in an input layer comprising 65 neurons and a single hidden layer with 64 neurons. The number of neurons in the output layer remains constant at six, as these six neurons represent the states of the subsequent time step. The dropout and the learning rate are very small, with a value of 5^{-5} and 7^{-5}, respectively. This may result in a bad fit. The activation function employed is ReLU, the epochs are set to 60, and the batch size is designated as 16. This results in a MSE of 0.0028. Considering the initial MSE of 0.0329 before the optimization, this corresponds to a significantly improved performance, reducing the error by approximately 91.48 %.

As demonstrated in Figure 3, the performance of the NARX model for the test data set is illustrated. The NARX model script includes supplementary figures that facilitate further investigation of its performance. The figure illustrates the model's satisfactory performance, exhibiting neither overfitting nor underfitting.

E. Conformalized Quantile Regression (Elisabeth Bünz)

To quantify uncertainty in the NARX model predictions, Conformalized Quantile Regression (CQR) was applied. This method allows for the construction of prediction intervals by estimating conditional quantiles for each output variable. Specifically, quantile regressors were trained to predict the 10th (q10) and 90th (q90) percentiles, enabling the generation of 80 % prediction intervals.

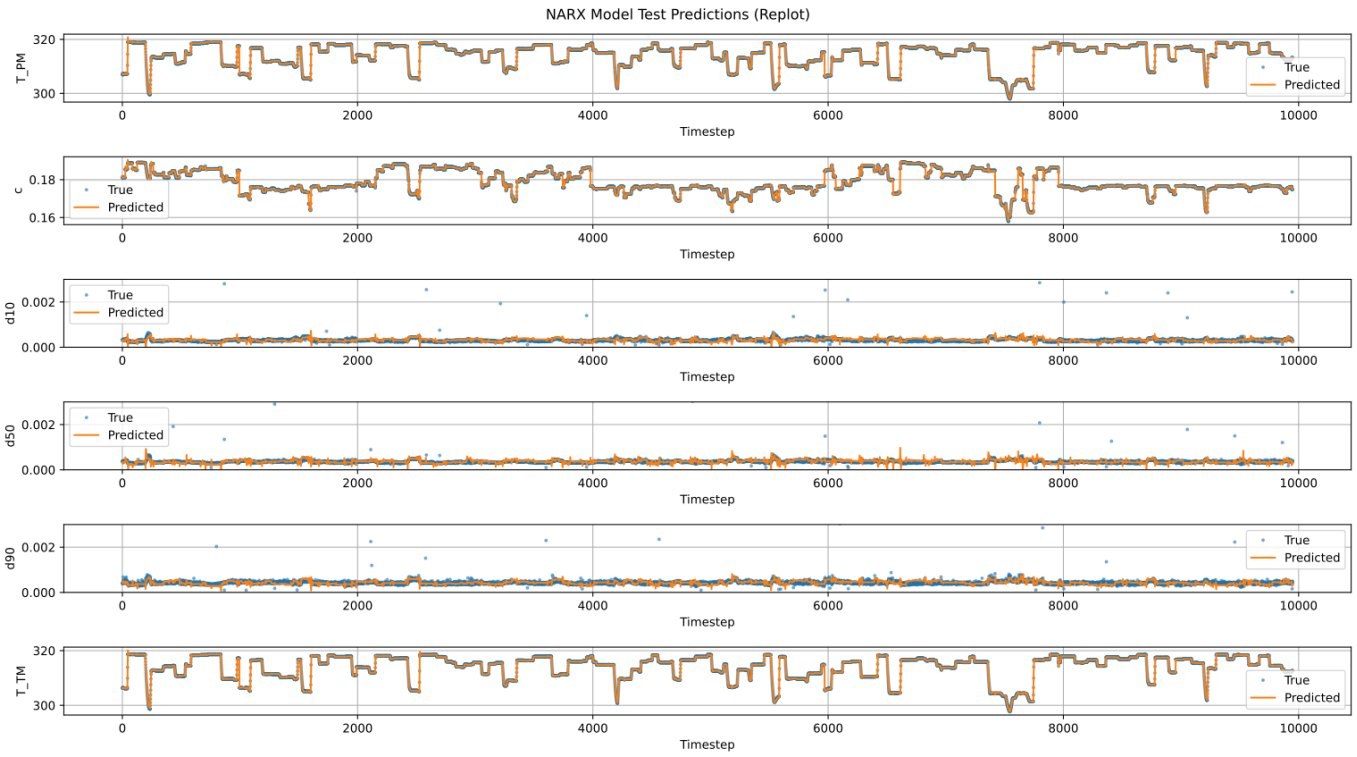

Fig. 3. Performance of the optimized NARX model for the test data set. The d10, d50 and d90 plots are zoomed in for better visibility, but some outliers fall outside the scale.

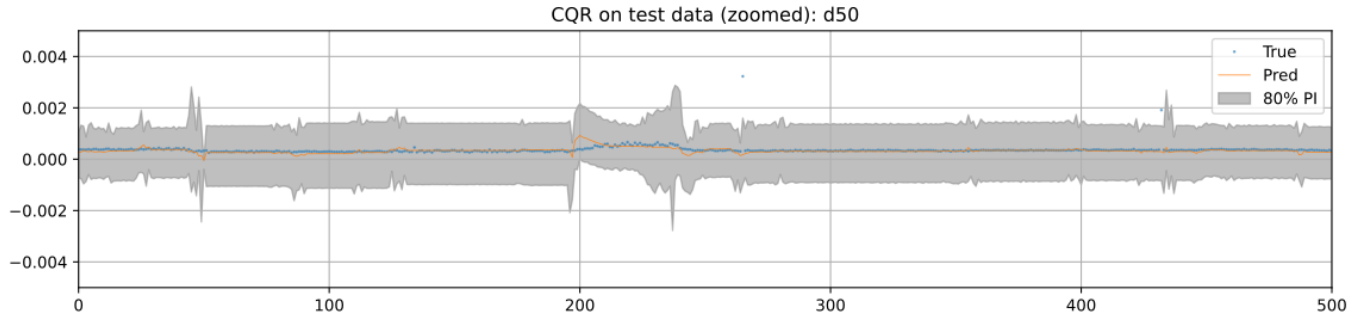

Fig. 4. Exemplary performance of the Conformalized Quantile Regression for the d50 state variable. The plot is zoomed in for better visibility of the effects.

A key advantage of the implemented pipeline is the modularity of the NARX codebase, which was reused for CQR with minimal adjustments. The same architecture, data preprocessing routines, lag structure, and training strategy were applied to the quantile regression models. This reuse ensures consistency and facilitates implementation. The only difference is, that the quantile regression models are trained on the respective quantiles of the residuals data set using the pinball loss function. To avoid computational expensive re-tuning, the hyperparameters obtained during NARX optimization were directly reused for all quantile models. This decision is justified by the fact that the residual dataset exhibits similar dynamics and structure as the original NARX training set. Consequently, it is reasonable to assume that the hyperparameters that performed well in the NARX model will also yield satisfactory performance in the quantile regressors. While a dedicated Bayesian optimization script for CQR was developed, it was not executed due to computational constraints. Training two separate models (q10 and q90) for each output variable substantially increases runtime and memory requirements. With six output variables, this results in 12 independent neural networks, making a full hyperparameter search impractical within the available compute resources. Nonetheless, the structure of the script allows for future execution and integration of automated hyperparameter tuning, should computational capacity allow.

Conformational analysis is performed as specified in Romano, 2019 [1]. The resulting performance of the CQR is exemplary illustrated for d50 in Figure 4. Of course, the script

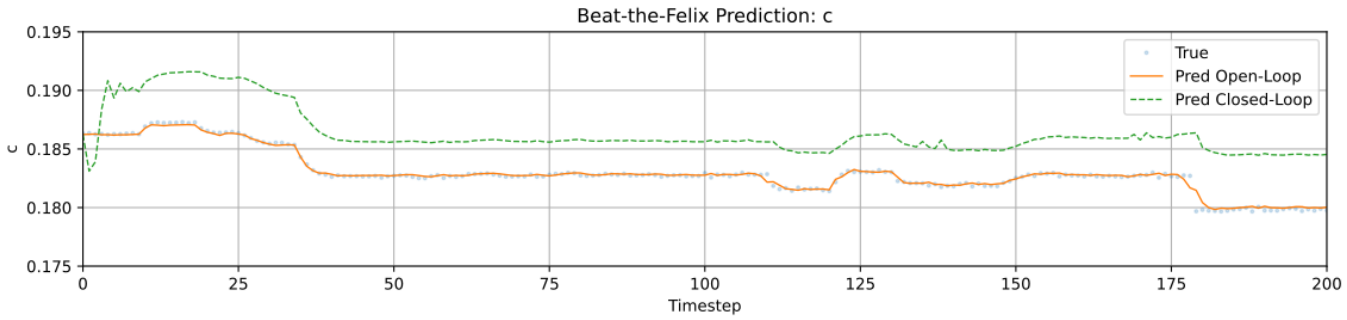

Fig. 5. Predicted concentration profile for the provided Beat-The-Felix trajectory in open- and closed-loop mode compared to the true values. Zoomed for the first 200 timesteps to show the initial sensibility.

generates the same diagrams for the other states. In addition, diagrams with arbitrary time intervals on the x-axis can be generated.

As illustrated in Figure 4, there are several outliers observed in the CQR. This discrepancy can be attributed to the presence of noise in the original data, resulting in a fairly high residual error between the predicted value by the NARX model and the true value. The following table II lists the results of the CQR for the test dataset. It is imperative to acknowledge that the conformalization process is executed employing the validation dataset. Consequently, the coverage of the CQR for the test dataset may deviate from the targeted 80 % threshold.

TABLE II
RESULTS OF THE CONFORMALIZED QUANTILE REGRESSION FOR THE TEST DATASET.

State	Coverage	Interval width
T_{PM}	0.443	0.133
T_{TM}	0.512	0.134
c	0.818	0.001
$d10$	0.772	0.001
$d50$	0.973	0.002
$d90$	0.763	0.001

As can be seen in Table II, coverage close to the threshold can be achieved for concentration and diameter. Small differences here are due to many similarly sized residuals, which can be seen from the small interval sizes. For temperatures, coverage close to the threshold cannot be achieved, which is due to the significantly larger absolute fluctuations in the temperature measurements in contrast to the small absolute fluctuations in the residuals. The combination of accurate predictions of the NARX model and fairly large absolute fluctuations in the state variable make it difficult to train good quantile regressors. It would be possible to improve performance through hyperparameter optimization.

F. Beat-The-Felix (Gina Lüker)

The final evaluation of the NARX model was conducted using the Beat-The-Felix (BTF) framework. This benchmark assesses the model's ability to predict long trajectories under realistic conditions and tests both open-loop and closed-loop performance. One visible effect of closed-loop prediction is an initial error (see Figure 5). The initial prediction error in closed-loop mode occurs because the model, although given correct lagged inputs, was only trained on perfect, ground-truth trajectories. As a result, it is not calibrated to produce stable predictions from the very first step when operating autonomously. Since for the closed loop only one initial lagged state is given, the first predictions of the model are quite sensitive. The errors produced in the first few predictions are carried forward continuously, as the subsequent predictions are based on the incorrect initial predictions. This effect is illustrated in Figure 5, which shows the predicted concentration profile in open- and closed-loop mode compared to the true values.

Nevertheless, the NARX model obtained through the project achieves similar or better performance on the provided trajectory. The exact values can be found in Table III.

TABLE III
MEAN PERFORMANCE METRICS (MSE, MAE) FOR THE OBTAINED NARX MODEL ON THE PROVIDED BEAT-THE-FELIX TRAJECTORY FOR OPEN- AND CLOSED-LOOP MODE.

State	Open Loop		Closed Loop	
	MSE	MAE	MSE	MAE
c	5.355e-08	1.231e-04	2.032e-05	4.380e-03
T_{PM}	6.048e-03	6.165e-02	6.756e-01	7.743e-01
$d50$	6.226e-09	3.837e-05	1.265e-05	3.536e-03
$d90$	1.538e-06	9.721e-05	1.643e-06	3.774e-04
$d10$	7.667e-07	8.533e-05	7.271e-06	2.533e-03
T_{TM}	3.915e-03	5.081e-02	5.418e-01	6.771e-01

The results show that the model maintains a high level of accuracy across all targets, even in closed-loop mode. Although closed-loop errors are naturally higher due to error propagation, they remain within acceptable bounds, indicating the model's robustness for long-horizon forecasting tasks.

REFERENCES

[1] Y. Romano, E. Patterson, and E. J. Candès, "Conformalized quantile regression," *arXiv preprint arXiv:1905.03222* , 2019. [Online]. Available: https://arxiv.org/abs/1905.03222